Самостоятельная работа

Тема: Сложные запросы к данным в PostgreSQL

Цель работы

Научиться выполнять сложные SQL-запросы к готовой базе данных: использовать фильтрацию, сортировку, объединение таблиц, агрегатные функции, группировку, HAVING, FILTER, подзапросы, CTE и VIEW.

Порядок выполнения

1. Создайте новую базу данных в PostgreSQL.
2. Выполните скрипт создания таблиц.
3. Выполните скрипт заполнения таблиц.
4. Выполните все задания ниже.
5. После каждого выполненного запроса:
   ставьте текст запроса в отчёт;
   сделайте скриншот результата;
   добавьте краткий комментарий о том, что делает запрос.

Что нужно сдать

SQL-файл со всеми запросами;
отчёт в Word или PDF;
скриншоты результатов;
комментарии к каждому запросу.

Задания

Блок 1. Фильтрация и сортировка

1. Вывести всех клиентов из Москвы.
2. Вывести все товары категории Electronics.
3. Найти товары дороже 5000 рублей.
4. Вывести оплаченные заказы после 2026-01-01.
5. Найти товары, название которых содержит слово Ноутбук или Смартфон.
6. Вывести 5 самых дорогих товаров.
7. Реализовать постраничный вывод товаров: вторая страница по 4 товара.

Блок 2. JOIN

8. Вывести список заказов: номер заказа, дата заказа, статус, имя клиента.
9. Вывести состав заказов: номер заказа, название товара, количество, цена позиции.
10. Найти всех клиентов, которые не сделали ни одного заказа.
11. Вывести все товары и количество раз, которое они встречались в заказах, включая товары без продаж.

Блок 3. Агрегация и группировка

12. Посчитать общее количество заказов в таблице orders.
13. Посчитать общую сумму всех оплаченных заказов.
14. Вывести количество заказов по каждому клиенту.
15. Вывести среднюю сумму заказа по каждому клиенту.
16. Найти клиентов, у которых больше 2 заказов.
17. Вывести категории товаров и максимальную цену в каждой категории.

Блок 4. FILTER

18. Для каждого клиента посчитать:
    общее число заказов;
    число оплаченных заказов;
    число отменённых заказов.
19. Для каждого клиента вывести отдельно:
    сумму оплаченных заказов;
    сумму отменённых заказов.

Блок 5. Подзапросы и EXISTS

20. Найти клиентов, у которых есть хотя бы один оплаченный заказ.
21. Найти товары, которые ни разу не встречались в заказах.
22. Вывести клиентов, сумма заказов которых больше средней суммы заказов по всем клиентам.

23. Найти заказы, в которых есть товары категории Electronics.

Блок 6. CTE

24. С помощью WITH создать временный набор оплаченных заказов и вывести по ним сумму заказов по клиентам.
25. С помощью CTE найти клиентов, у которых более 1 оплаченного заказа, и вывести их средний чек.
26. Разбить сложный запрос на несколько этапов через несколько CTE.

Блок 7. VIEW

27. Создать представление v_order_details, содержащее:
    номер заказа;
    имя клиента;
    дату заказа;
    статус;
    товар;
    количество;
    цену позиции.
28. Выполнить запрос к представлению и вывести все товары, купленные конкретным клиентом.

Лекция прямо приводит пример витрины заказов через VIEW, то есть это хорошее завершающее практическое задание.

Блок 8.Аналитика

29. Составить итоговый аналитический запрос, который выводит:
    имя клиента;
    количество оплаченных заказов;
    общую сумму оплаченных заказов;
    средний чек по оплаченных заказам;

Оставить только тех клиентов, у которых сумма оплаченных заказов больше 5000, и отсортировать результат по убыванию суммы.

Шаблон комментария

После каждого задания пишете комментарий по шаблону:

Комментарий:

В запросе используются таблицы ....
Запрос выводит ....
Условие WHERE / JOIN / HAVING нужно для того, чтобы ....
Результат показывает ....

Пример

Комментарий:

В запросе используются таблицы clients и orders.
Запрос выводит клиентов, у которых есть оплаченные заказы.
Связь между таблицами выполняется по полям clients.id = orders.client_id.
Условие по статусу позволяет оставить только оплаченные заказы.

Шаблон отчёта для сдачи

Отчёт по самостоятельной работе

Тема: Сложные запросы к данным в PostgreSQL
Студент: _____
Группа: _____
Дата: _____

Задание №1

SQL-запрос:

-- сюда вставляется запрос

Скриншот результата:

вставить скриншот(уникальный)

Комментарий:

краткое пояснение студента

Задание №2
SQL-запрос:
-- сюда вставляется запрос
Скриншот результата:
вставить скриншот
Комментарий:
краткое пояснение студента
Такой же блок повторяется для всех заданий.
Итоговый вывод (Саморефлексия)
Какие запросы вызвали наибольшую трудность?
Что получилось лучше всего?
Какие темы нужно повторить?

Доп инфа (Чек лист самопроверки)
мини-памятка по типовым ошибкам

забытый ON ведёт к декартову произведению;
неверный ключ в JOIN даёт некорректные результаты;
фильтр по правой таблице в WHERE может “сломать” LEFT JOIN;
при использовании LIMIT нужен ORDER BY;
фильтры строк и фильтры групп надо разделять на WHERE и HAVING.

PS

Скрипт создания базы

```
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS clients;

CREATE TABLE clients (
   id SERIAL PRIMARY KEY,
   full_name VARCHAR(100) NOT NULL,
   city VARCHAR(100) NOT NULL,
   email VARCHAR(120) UNIQUE,
   phone VARCHAR(30),
   registered_at DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE products (
   id SERIAL PRIMARY KEY,
   title VARCHAR(150) NOT NULL,
   category VARCHAR(100) NOT NULL,
   price NUMERIC(10,2) NOT NULL CHECK (price > 0),
   stock INTEGER NOT NULL CHECK (stock >= 0)
);

CREATE TABLE orders (
   id SERIAL PRIMARY KEY,
   client_id INTEGER NOT NULL,
   order_date DATE NOT NULL,
   status VARCHAR(20) NOT NULL CHECK (status IN ('new', 'paid', 'canceled', 'shipped')),
   total_amount NUMERIC(10,2) NOT NULL CHECK (total_amount >= 0),
   CONSTRAINT fk_orders_client
      FOREIGN KEY (client_id) REFERENCES clients(id)
      ON DELETE CASCADE
);
```

```
CREATE TABLE order_items (
   id SERIAL PRIMARY KEY,
   order_id INTEGER NOT NULL,
   product_id INTEGER NOT NULL,
   quantity INTEGER NOT NULL CHECK (quantity > 0),
   price NUMERIC(10,2) NOT NULL CHECK (price > 0),
   CONSTRAINT fk_order_items_order
      FOREIGN KEY (order_id) REFERENCES orders(id)
      ON DELETE CASCADE,
   CONSTRAINT fk_order_items_product
      FOREIGN KEY (product_id) REFERENCES products(id)
      ON DELETE CASCADE
);
```

Скрипт заполнения базы

```
INSERT INTO clients (full_name, city, email, phone, registered_at) VALUES
('Иван Петров', 'Москва', 'ivan@example.com', '+79990000001', '2025-09-01'),
('Мария Соколова', 'Санкт-Петербург', 'maria@example.com', '+79990000002', '2025-10-12'),
('Алексей Иванов', 'Казань', 'alex@example.com', '+79990000003', '2025-11-03'),
('Ольга Смирнова', 'Москва', 'olga@example.com', '+79990000004', '2025-12-21'),
('Дмитрий Кузнецов', 'Екатеринбург', 'dmitry@example.com', '+79990000005', '2026-01-15'),
('Елена Орлова', 'Новосибирск', 'elena@example.com', '+79990000006', '2026-01-20'),
('Сергей Волков', 'Москва', 'sergey@example.com', '+79990000007', '2026-02-01'),
('Анна Морозова', 'Самара', 'anna@example.com', '+79990000008', '2026-02-10');

INSERT INTO products (title, category, price, stock) VALUES
('Ноутбук Lenovo IdeaPad', 'Electronics', 65000.00, 12),
('Смартфон Samsung Galaxy', 'Electronics', 48000.00, 18),
('Беспроводная мышь Logitech', 'Electronics', 2500.00, 40),
('Клавиатура механическая', 'Electronics', 7200.00, 15),
('Офисное кресло', 'Furniture', 13500.00, 7),
('Письменный стол', 'Furniture', 18900.00, 5),
('Рюкзак городской', 'Accessories', 3200.00, 25),
('Термокружка', 'Accessories', 1200.00, 30),
('Монитор 27 дюймов', 'Electronics', 22100.00, 10),
('Настольная лампа', 'Furniture', 2900.00, 14),
('Наушники Bluetooth', 'Electronics', 5400.00, 20),
('Ежедневник', 'Office', 700.00, 50);

INSERT INTO orders (client_id, order_date, status, total_amount) VALUES
(1, '2026-01-05', 'paid', 67500.00),
(1, '2026-01-20', 'shipped', 3200.00),
(2, '2026-01-11', 'paid', 61500.00),
(2, '2026-02-02', 'canceled', 2500.00),
(3, '2026-01-18', 'paid', 24300.00),
(3, '2026-02-15', 'paid', 8600.00),
(3, '2026-02-20', 'shipped', 1200.00),
(4, '2026-01-25', 'new', 13500.00),
(5, '2026-02-01', 'paid', 18900.00),
(5, '2026-02-17', 'canceled', 5400.00),
(7, '2026-02-05', 'paid', 72400.00),
(7, '2026-02-25', 'paid', 6100.00);
```

```
INSERT INTO order_items (order_id, product_id, quantity, price) VALUES
(1, 1, 1, 65000.00),
(1, 3, 1, 2500.00),

(2, 7, 1, 3200.00),

(3, 2, 1, 48000.00),
(3, 11, 1, 5400.00),
(3, 8, 1, 1200.00),
(3, 3, 1, 2500.00),
(3, 12, 5, 700.00),

(4, 3, 1, 2500.00),

(5, 9, 1, 22100.00),
(5, 8, 1, 1200.00),
(5, 12, 1, 700.00),
(5, 7, 1, 3200.00),

(6, 4, 1, 7200.00),
(6, 8, 1, 1200.00),
(6, 12, 3, 700.00),

(7, 8, 1, 1200.00),

(8, 5, 1, 13500.00),

(9, 6, 1, 18900.00),

(10, 11, 1, 5400.00),

(11, 1, 1, 65000.00),
(11, 11, 1, 5400.00),
(11, 8, 1, 1200.00),
(11, 12, 1, 700.00),

(12, 3, 1, 2500.00),
(12, 10, 1, 2900.00),
(12, 12, 1, 700.00);
```